CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

CO3 CASE STUDY – COMPLETE SOLVED ANSWERS

Submitted by : P.VIGNESH

Register Number: 192525013

Department / Class: AIML

Course: Design and Analysis of Algorithms

Case background
KartIQ handles approximately 10,000,000 orders per day and ranks 100,000 sellers. Four modules require redesign: stable timestamp sorting, frequent seller-score ranking, first-match timestamp lookup, and 1,024 × 1,024 recommendation matrix multiplication. The answers below follow parts (a)–(f) in the provided assessment.

(a) Understand the case

Module	Requirement and key issue	Poor algorithm consequence
1. Orders	Sort 10 million orders by timestamp; stable ties preserve arrival order; memory may be limited.	Unstable sort breaks audit reconciliation; quadratic sort misses nightly deadline.
2. Leaderboard	Rank 100,000 scores frequently; nearly sorted input; first-element pivot currently used.	First-pivot Quick Sort can repeatedly split 0 and n−1, freezing updates.
3. Lookup	Find first order with timestamp among sorted records with duplicates.	Linear search takes up to 10 million checks; ordinary binary search may return a later duplicate.
4. Recommendations	Multiply dense n×n matrices with n=1,024; memory and constant overhead matter.	Naive matrix multiplication requires cubic work; Strassen without tuning can waste memory/time.

(b) Module 1 – Stable Merge Sort

Divide, conquer and combine
Divide the orders into two nearly equal halves; recursively sort both halves by timestamp; merge in ascending timestamp order. When timestamps tie, choose the order from the LEFT half first (≤), preserving original arrival order. This is stable because the two halves preserve their original relative order.

```
merge_sort(A):
    if len(A) <= 1: return A
    mid = len(A)//2
    L = merge_sort(A[:mid]); R = merge_sort(A[mid:])
    return stable_merge(L,R)  # on tie, choose L first
```

Complete merge trace

Stage	Left / right	Result
Input	[(105,O1),(102,O2),(105,O3)] | [(101,O4),(102,O5),(103,O6)]	Split into two halves
Left submerge 1	[(102,O2)] + [(105,O3)]	[(102,O2),(105,O3)]
Left submerge 2	[(105,O1)] + [(102,O2),(105,O3)]	[(102,O2),(105,O1),(105,O3)]
Right submerge 1	[(102,O5)] + [(103,O6)]	[(102,O5),(103,O6)]
Right submerge 2	[(101,O4)] + [(102,O5),(103,O6)]	[(101,O4),(102,O5),(103,O6)]
Final merge	Sorted left + sorted right	[(101,O4),(102,O2),(102,O5), (103,O6),(105,O1),(105,O3)]

Stability evidence: O2 arrives before O5 among timestamp 102, and O1 arrives before O3 among timestamp 105. Both pairs remain in that order. The full recursion also contains singleton base cases; these require no merge comparisons.

Recurrence, comparisons and memory

$T(n)=2T(n/2)+\Theta(n)$, $T(1)=\Theta(1)$. By Master Theorem case 2, $T(n)=\Theta(n \log_2 n)$. A standard worst-case comparison bound for n keys is $n\lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil}+1$. For n=10,000,000, $\lceil \log_2 n \rceil=24$: 240,000,000−16,777,216+1 = 223,222,785 comparisons at most. A rough $n \log_2 n$ estimate is about 232.53 million operations (not an exact comparison count).

Merge Sort typically needs $\Theta(n)$ extra storage for an auxiliary merge buffer, plus $\Theta(\log n)$ recursion stack. If orders exceed RAM, use external merge sort: sort memory-sized runs, write them to disk, and perform multiway merges with buffered sequential I/O. Preserve original arrival order within equal timestamps, e.g. by stable merges and an arrival-sequence tie-breaker across runs.

Executable Python example

```
def merge_orders(orders):
    if len(orders) <= 1: return orders[:]
    mid = len(orders)//2
    L, R = merge_orders(orders[:mid]), merge_orders(orders[mid:])
    i = j = 0; out = []
    while i < len(L) and j < len(R):
        if L[i][0] <= R[j][0]: out.append(L[i]); i += 1
        else: out.append(R[j]); j += 1
    return out + L[i:] + R[j:]
```

(c) Module 2 – Quick Sort for live seller leaderboard

Using the first element 42 as pivot and the simple “less than pivot / greater than pivot” partition convention, scores [42,17,63,8,55,29] split into left [17,8,29], pivot [42], right

[63,55]. The exact in-place array after one partition depends on the partition scheme, but the sets of scores on each side are as shown.

Partition element	Value(s)
Pivot	42
Left (<42)	17, 8, 29
Right (>42)	63, 55

On ascending input, choosing the first element produces 0 and n−1 partitions every time; descending input similarly produces extreme partitions. For 100,000 nearly ordered sellers, the worst-case number of pivot-to-element comparisons is $n(n-1)/2 = 4{,}999{,}950{,}000$. This is a worst-case estimate, not a guaranteed count for every nearly sorted input.

Best/ideal balanced recurrence: $T(n)=2T(n/2)+\Theta(n)=\Theta(n \log n)$. Worst/unbalanced recurrence: $T(n)=T(n-1)+\Theta(n)=\Theta(n^2)$. The balanced comparison scale for n=100,000 is roughly $n \log_2 n \approx 1{,}660{,}964$ (constant factors depend on partitioning).

Improved pivot: randomised pivot or median-of-three (first, middle, last) reduces the likelihood of extreme partitions on sorted data. Use three-way partitioning when many scores tie. For a production guarantee, introsort (Quick Sort that switches to Heap Sort on excessive recursion depth) limits worst-case time to O(n log n). Since scores change only slightly, a stable adaptive Timsort is also a strong practical alternative; benchmark it for frequent refreshes.

```
def quicksort_random(a):
    import random
    if len(a) <= 1: return a[:]
    pivot = random.choice(a)
    left  = [x for x in a if x < pivot]
    equal = [x for x in a if x == pivot]
    right = [x for x in a if x > pivot]
    return quicksort_random(left) + equal + quicksort_random(right)
```

(d) Module 3 – Binary Search for first timestamp

Binary Search works only because Module 1 sorts timestamps into nondecreasing order. At each step compare the target timestamp to the middle timestamp; discard half of the remaining range. To return the first duplicate, use a lower-bound search: when A[mid].timestamp ≥ target, keep searching left, even when equal.

```
def first_order_at(orders, target):
    lo, hi = 0, len(orders)
    while lo < hi:
        mid = (lo + hi)//2
```

```
        if orders[mid][0] < target: lo = mid + 1
        else: hi = mid
    return lo if lo < len(orders) and orders[lo][0] == target else -1
```

For a halving search, $T(n)=T(\lceil n/2 \rceil)+\Theta(1)=\Theta(\log_2 n)$. At $n=10,000,000$, $\lceil \log_2(10,000,001) \rceil=24$, so at most 24 midpoint comparisons for this lower-bound loop; a final equality check verifies the candidate. Thus there are at most 25 timestamp comparisons if the final equality test is counted separately. The result for timestamp 102 is index 1 (order O2), not index 2 (O5).

Target	Expected result
102	Index 1: (102,O2), first duplicate
105	Index 4: (105,O1), first duplicate
999	−1, not found

(e) Module 4 – Strassen's matrix multiplication

Given A=[[1,7],[3,5]] and B=[[1,3],[7,5]], partition into scalar blocks: a=1,b=7,c=3,d=5; e=1,f=3,g=7,h=5.

M1=(a+d)(e+h)=(1+5)(1+5)=36
M2=(c+d)e=(3+5)*1=8
M3=a(f-h)=1*(3-5)=-2
M4=d(g-e)=5*(7-1)=30
M5=(a+b)h=(1+7)*5=40
M6=(c-a)(e+f)=(3-1)*(1+3)=8
M7=(b-d)(g+h)=(7-5)*(7+5)=24

Reconstruct C11=M1+M4−M5+M7=36+30−40+24=50; C12=M3+M5=−2+40=38; C21=M2+M4=8+30=38; C22=M1−M2+M3+M6=36−8−2+8=34. Thus C=[[50,38],[38,34]]. Conventional multiplication independently gives the same result: 1×1+7×7=50; 1×3+7×5=38; 3×1+5×7=38; 3×3+5×5=34.

Recurrence $T(n)=7T(n/2)+\Theta(n^2)$, so by Master Theorem $T(n)=\Theta(n^{\log_2 7})\approx\Theta(n^{2.807})$. At $n=1,024=2^{10}$, conventional multiplication uses $n^3=1,073,741,824$ scalar multiplications. Pure recursive Strassen uses $7^{10}=282,475,249$ scalar multiplications, saving 791,266,575 (about 73.69%). This compares multiplication counts only; Strassen also performs many additions and temporary allocations.

Strassen can be slower for small submatrices because of recursive overhead, extra additions, memory traffic, cache locality, and temporary matrices. Floating-point error can also differ from conventional multiplication. A practical hybrid applies Strassen above a measured threshold (e.g. 64 or 128, to be tuned), and uses a highly optimised conventional blocked/BLAS matrix multiplication below that threshold. Test runtime and numerical accuracy on realistic 1,024×1,024 matrices before deploying.

(f) Final design and justification

Module	Recommended design	Time / extra memory	Trade-off and when to change
1	Stable Merge Sort; external multiway merge for out-of-core orders	O(n log n) / O(n) buffer in RAM	Stable and predictable, but uses memory. Switch to external merging if RAM is insufficient.
2	Introsort with robust pivot and three-way partition; benchmark adaptive stable Timsort	O(n log n) worst for introsort / O(log n) stack typical	Robustness versus implementation complexity. Prefer Timsort if updates are consistently nearly sorted and stability matters.
3	Lower-bound Binary Search on timestamp-sorted orders	O(log n) / O(1) iterative	Requires sorted input. Use an indexed database structure if orders change continuously.
4	Hybrid Strassen + optimised blocked matrix multiplication	O(n^2.807) asymptotic / O(n²) extra workspace typical	Fewer multiplies but more additions/ memory. Use conventional BLAS if measurements or accuracy favour it.

System flow and integration

Incoming orders → stable timestamp sorting → consolidated audit records → lower-bound order lookup. Seller score updates → robust ranking sort → refreshed leaderboard. Segment-feature preferences × feature-product weights → hybrid matrix multiplication → recommendation scores. Each module should be benchmarked independently and monitored under peak load.

Validation tests and expected results

Test	Expected result
Stable order sort	[(101,O4),(102,O2),(102,O5),(103,O6),(105,O1),(105,O3)]
Quick Sort sample	[8,17,29,42,55,63]
First timestamp 102	Index 1, order O2
First timestamp 105	Index 4, order O1
Matrix multiplication	[[50,38],[38,34]]

Conclusion

KartIQ should use stable sorting for auditable orders, a robust and adaptive leaderboard sorter, lower-bound Binary Search for duplicate timestamps, and a threshold-tuned hybrid for large matrix multiplication. The design choices follow the case requirements and address performance without sacrificing correct ordering or first-match semantics. The numerical estimates are theoretical counts; practical throughput and latency must be measured on production-like data.